Implementation Strategy

Multilingual NLU System for Indic Languages

This document outlines a structured, 4-week execution strategy to develop a robust natural language understanding (NLU) system. The pipeline is optimized to run within standard free-tier GPU constraints (e.g., Kaggle/Google Colab) and focuses heavily on cross-lingual representation learning to bridge the gap between high-resource and low-resource Indian languages.

Week 1: Data Curation & Preprocessing Pipeline

Objective: Secure datasets, clean raw text, and establish a localized preprocessing pipeline.

- **Dataset Acquisition:** Download the *Samanantar* parallel corpus to aid in cross-lingual alignment. Source labeled datasets for sentiment (e.g., AI4Bharat's IndicSentiment) and use translations for intent datasets if native low-resource datasets are scarce.
- **Text Processing:** Implement custom text cleaning modules to handle unstructured domains (social media emojis, hashtags, URLs). Use tokenizers optimized for morphologically rich Indian languages (e.g., IndicNLP Library or SentencePiece).
- **Environment Setup:** Initialize the workspace. Ensure Hugging Face `transformers` and `datasets` libraries are installed, and configure standard data loaders.

Week 2: Model Selection & Baseline Training

Objective: Establish the transformer architecture and train a baseline model within GPU constraints.

- **Model Selection:** Load a pre-trained cross-lingual model. **XLM-RoBERTa (XLM-R)** or Google's **MuRIL** are highly recommended for handling regional morphological complexities.
- **Architecture Design:** Design a single shared transformer encoder with two separate classification heads (Multi-Task Learning): one for Sentiment (Positive, Negative, Neutral) and one for Intent (Inquiry, Opinion, Factual).
- **Resource Optimization:** Implement Mixed Precision Training (FP16) and Gradient Accumulation to maintain large batch sizes without exceeding the 15–16GB memory limits of free-tier GPUs.

Week 3: Fine-Tuning, Metrics & Evaluation

Objective: Optimize model performance, extract probability scores, and evaluate cross-lingual transfer.

- **Hyperparameter Tuning:** Unfreeze all layers and perform full fine-tuning with a low learning rate (e.g., 2e-5) using a scheduler with warmup.
- **Confidence Scoring:** Configure the classification output layers to apply a Softmax activation, converting raw logits into structured probability and confidence scores.
- **Cross-Lingual Evaluation:** Test the model's zero-shot or few-shot transfer capabilities (e.g., train predominantly on Hindi data and evaluate on Bengali or Marathi test sets).
- **Metric Calculation:** Compute macro and micro F1-scores and Accuracy across all target languages to identify performance disparities.

Week 4: Documentation & Final Reports

Objective: Finalize deliverables, focusing on code modularity, clarity, and analytical reporting.

- **Notebook Finalization:** Refactor the notebook to be highly modular. Separate cells into clear sections (Config, Data Loaders, Model Definition, Training Loop, Inference). Add comprehensive markdown comments.
- **Evaluation Report:** Compile accuracy and F1-score tables. Create visualizations such as confusion matrices or bar charts comparing performance on sentiment vs. intent across regional languages.
- **Analytical Summary:** Write a report analyzing cross-lingual transfer results, discussing how the tokenization handled agglutination, and how semantic gaps were bridged without direct translation.

Expected Project Deliverables

- A modular, well-documented notebook containing the complete end-to-end training and inference pipeline.
- An evaluation report demonstrating the model’s performance on the target languages using standard metrics (F1-score, accuracy).
- A brief analytical summary detailing cross-lingual transfer capabilities and the handling of linguistic complexities.